

CypherLens

An Interactive Demo for Evaluating and Diagnosing NL-to-Cypher Systems

Zahra Mohammadi · Ali Ansari · Longin Latecki · Eduard Dragut

Temple University, Philadelphia, PA, USA · PVLDB 19(12) · VLDB 2026, Boston



Code, data & demo
github.com/YasmeanMohammadi/CypherLens

Why execution accuracy falls short

C1 Metric brittleness

Strict result equality punishes queries that differ only in aliases, column order or variable names.
Gold returns 1937 under min(n.inception_year); the model returns 1937 as earliest_founding_year → EA = 0.

C2 No result normalization

Cypher returns nested, heterogeneous JSON, not flat tables.
n.name, n.gender vs. p.name AS founder, p.gender AS gender
hold identical data yet look mismatched; nulls, timeouts and empty results are unhandled.

C3 Error opacity & query blindness

A 0/1 signal never says why a query failed: wrong string literal? missing OPTIONAL MATCH? reversed traversal? Aggregate scores also hide whether failures sit in retrieval, aggregation or multi-hop reasoning.

C4 Binary scoring

A near-miss scores the same as nonsense. Asked for Czech-HQ CEOs, a query that returns the right CEOs plus company, start and end year still gets 0.

C5 Coincidental correctness

A structurally wrong query can return the right rows on this dataset – e.g. ignoring a player's Lakers tenure when filtering awards – and EA calls it correct, hiding a bug that surfaces when data changes.

7.75%

of queries on the Neo4j Twitter dataset were flagged as correct-by-coincidence candidates – scored as correct by execution accuracy despite semantic errors.

How CypherLens answers each challenge

- C1 · C2 Schema-aware normalization & alias alignment
- C3 Query-type classifier + Cypher-specific diagnoser
- C4 Graded Exact / Soft Match and Dice scores
- C5 Flags correct-by-coincidence (CBC) candidates

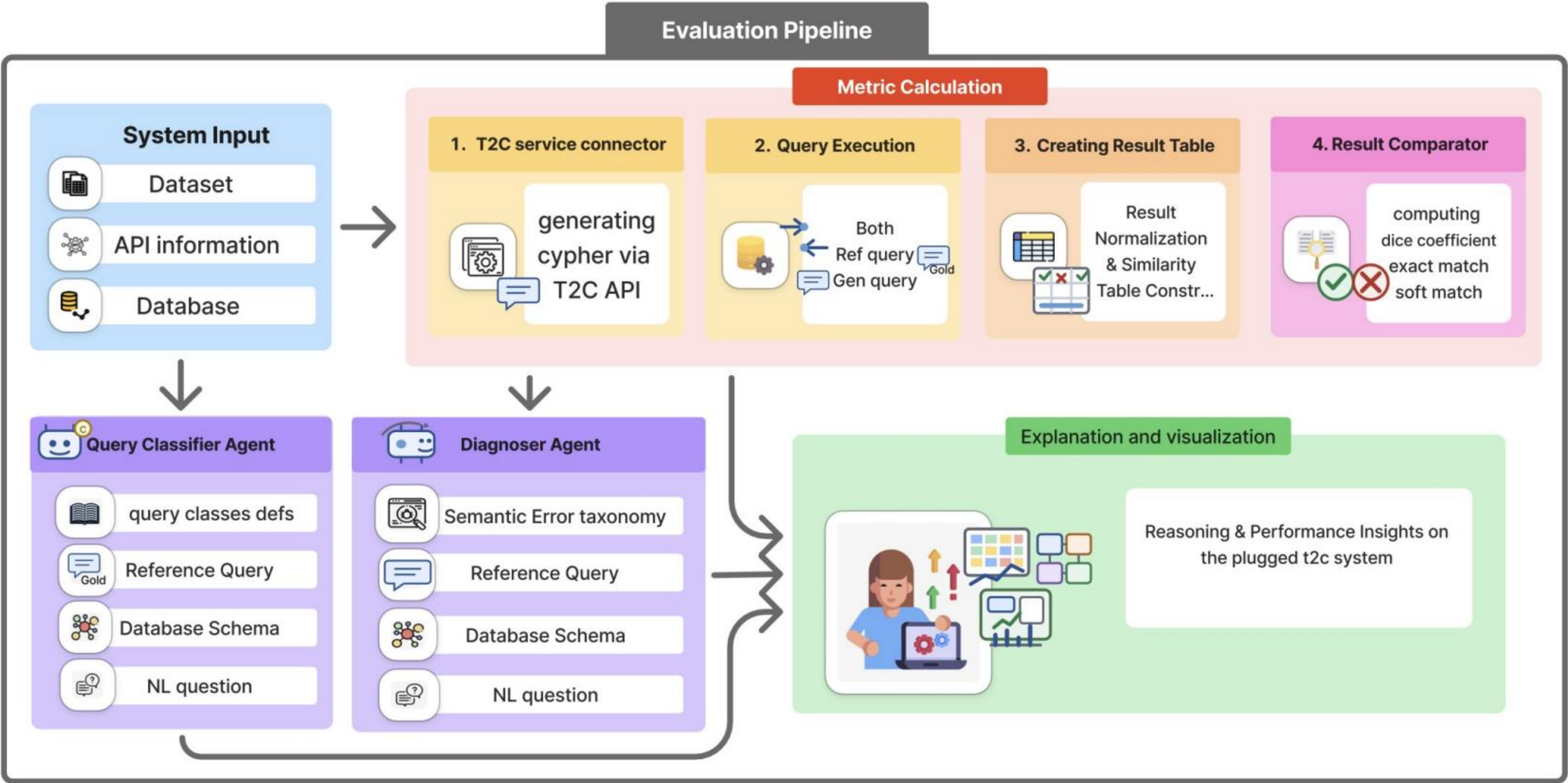
Demonstration scenarios

1 Setup & dataset-level evaluation

Point CypherLens at a dataset, a database and a T2C API; toggle the Diagnoser, Classifier and CBC inspection and pick an LLM backend. One click runs the full pipeline with streamed progress and logs, then opens the dashboard: graded scores, per-class outcomes and error distributions (e.g. simple retrieval solved, pattern-topology errors dominate multi-hop).

Takeaway CypherLens moves NL-to-Cypher evaluation beyond a 0/1 verdict: normalized results, graded metrics, query-class breakdowns and Cypher-native error labels let users debug single queries, compare systems and spot when execution accuracy masks semantic errors. **Next:** closing the loop with automatic refinement and benchmark-level validation of diagnostic accuracy.

System overview: a black-box evaluation pipeline



Inputs: NL questions with gold Cypher · Neo4j connection · API access to the target T2C system (no access to model internals). **Pipeline:** ① generate Cypher via the T2C API → ② execute generated and gold queries → ③ normalize results and build a Similarity Table → ④ compute Exact Match, Soft Match and Dice. Classifier and Diagnoser Agents run through a provider-agnostic LLM routing layer.

Inside CypherLens

1 Execution & schema-aware evaluation

Retrieves node labels, relationship types, property keys and directions from Neo4j, then aligns generated and gold projections through staged matching: exact columns → alias normalization → type-aware → schema-aware, with embedding alignment only as a fallback (e.g. sum(x)/size(collect(x)) ↔ avg(x)). The Result Comparator turns the table into Exact Match, Soft Match and Dice.

2 Query Type Classifier Agent

Assigns every instance to a canonical class – simple retrieval, aggregation, multi-hop traversal, path query – from the question, schema and gold query. Reuses dataset labels when they exist, so annotated and unannotated benchmarks both get per-class metric and error breakdowns.

3 Diagnoser Agent

Invoked whenever EM = 0. Reasons over question, schema, gold and generated queries plus sampled results, then emits a label from a hierarchical Cypher error taxonomy (projection & value, pattern topology, binding & scope, clause-level – e.g. Direction Mismatch, Wrong Hop Count, Implicit Condition Missing) with a confidence score and grounded rationale. Also audits EM = 1 cases for coincidental correctness.

4 Interactive visualization interface

Dataset level: aggregate metrics, per-class outcome bars and hierarchical error rings. Query level: question, gold and generated Cypher side by side with execution outputs, the similarity table, metric scores, class label and the diagnosis report.

What the dashboard reveals

0.352

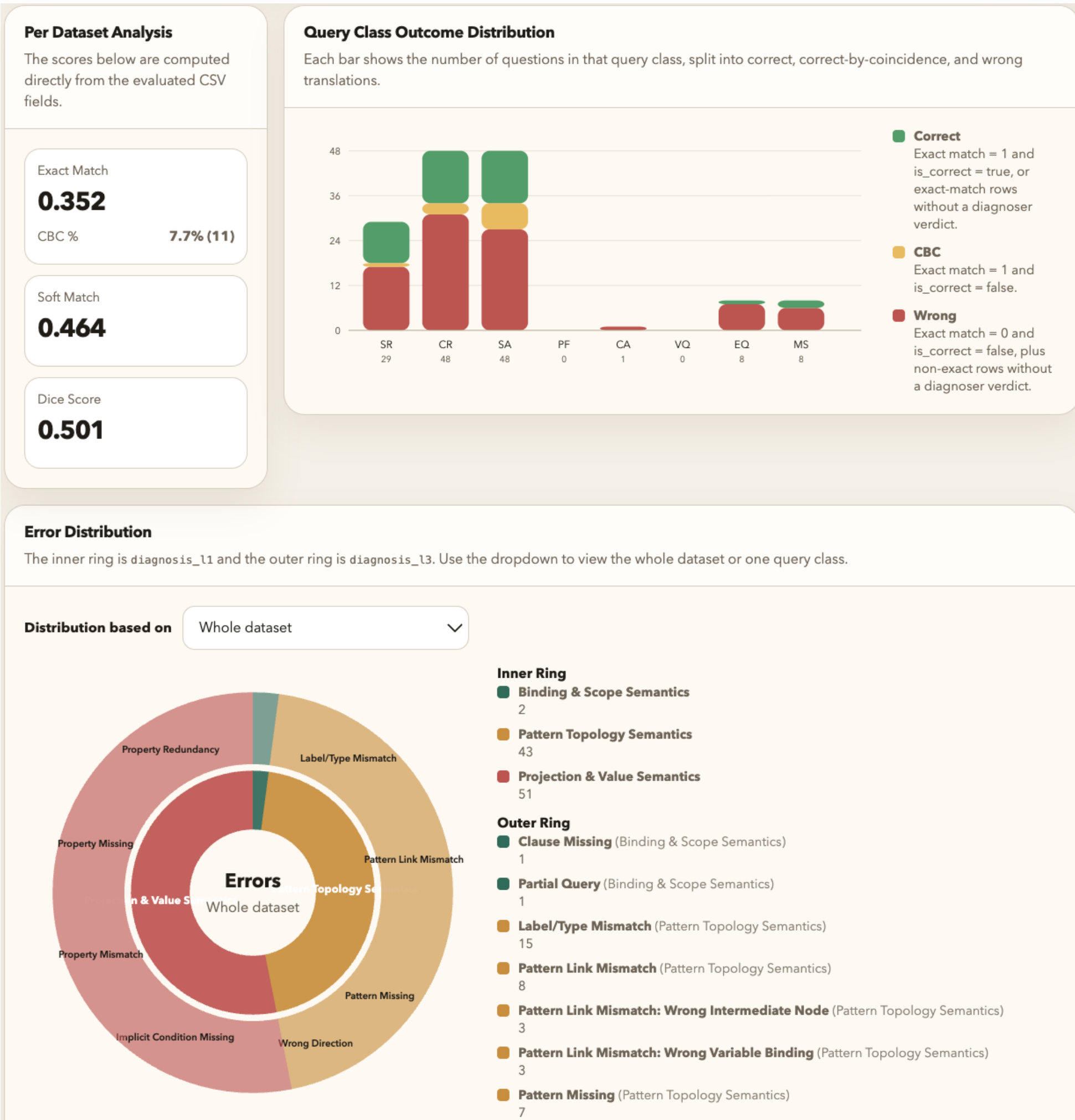
Exact Match

0.464

Soft Match

0.501

Dice score



Read the gap: Soft Match 0.464 vs. Exact Match 0.352 means many “failures” are near-misses, not wrong answers.

Where errors live: Projection & Value (51) and Pattern Topology (43) semantics dominate; label/type mismatch, pattern-link mismatch and missing patterns concentrate in multi-hop classes.

Hidden bugs: 11 EM = 1 queries (7.7%) are CBC candidates.

Demo setup: Text2Cypher Twitter subset on Neo4j · prompting-based T2C via webhook · agents on Gemini-2.5-Pro

2 Per-query debugging

Open any instance to see the question, gold and generated Cypher side by side with their results and the similarity table. The Diagnoser explains, for example, a one-hop relationship where a two-hop traversal was required (Wrong Hop Count); Soft Match 0.5 shows the core result is partially right where EA gives 0.

3 Correct-by-coincidence inspection

Filter to EM = 1 ∧ CBC-flagged. A generated query links employees straight to cities, skipping the intermediate project node the gold path requires – identical results on this data, but a Pattern Topology error the Diagnoser catches. About 7% of EM = 1 queries are flagged.

This work was supported in part by the U.S. NSF awards III-2107213, and ITE-2333789.